

Task 1: Formal State-Space Modeling for Uninformed Search (Problem Formulation)

Objective

Construct a formal **state-space representation** suitable for uninformed search algorithms.

Problem Description

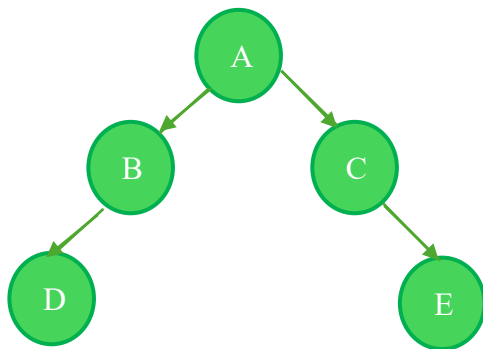
Model a deterministic, discrete search environment as a **directed graph**, where:

- Nodes represent **states**
- Edges represent **state transitions**
- Edge weights (if present) represent **step costs**

Students must clearly define:

- Initial State
- Goal State
- Successor Function
- Cost Function

Graph Representation



- **Initial State:** A
- **Goal State:** E

Implementation Requirements

- Represent the graph using an adjacency list
- Design the graph representation such that different uninformed search algorithms can be applied without modifying the underlying data structure. (**Define a function (or class)**)

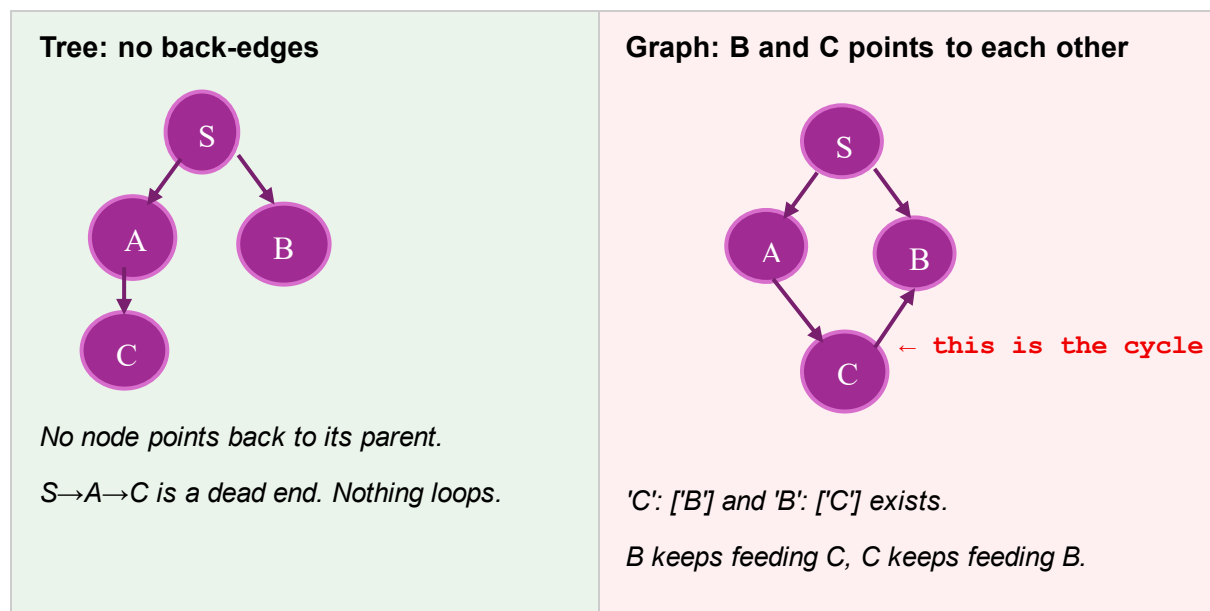
that encapsulates the search problem parameters, so that *any* uninformed search algorithm can operate on it without changing the problem definition.)

- Maintain separation between **problem definition** and **search logic**

Task 2: Tree vs Graph Search - The Fundamental Difference

The Two Structures

Both start from S and have the same nodes. The only difference is one back-edge.



tree	graph
<pre>tree = { 'S': ['A', 'B'], 'A': ['C'], 'B': [], 'C': [], }</pre>	<pre>graph = { 'S': ['A', 'B'], 'A': ['C'], 'B': ['C'], # ← B points to C 'C': ['B'], # ← C points to B }</pre>

Part 1: BFS on Tree (no visited set)

Run BFS on the tree from S, searching for goal C. Do NOT use a visited set.

Step-by-step queue trace

Step	Take from Queue	Add to Queue	Queue after this step
1	S	A, B	[A, B]
2	A	C	[B, C]
3	B	(none)	[C]
4	C	(none)	[] ← empty, BFS is done!

What you observe

Every node is visited exactly once: S, A, B, C.

BFS finishes cleanly. No problems at all.

Why? No node ever points back. Once we reach C, it has no neighbors.

There is simply no path that could loop.

Expected output when you run it

```
Visiting: S
```

```
Visiting: A
```

```
Visiting: B
```

```
Visiting: C
```

```
Goal found! Path: S → A → C
```

Part 2: BFS on Graph without visited set → infinite loop

Run the exact same BFS on the graph, **without** a visited set. Search for C.

Step-by-step queue trace

Step	Take from Queue	Add to Queue	Queue after this step
1	S	A, B	[A, B]
2	A	C	[B, C]
3	B	C	[C, C] ← C added twice!
4	C	B	[C, B] ← C puts B back
5	C	B	[B, B] ← C again!
6	B	C	[B, C] ← B puts C back
7	B	C	[C, C] ← same as step 3!
...	forever	...	queue never empties

What you observe

The queue never empties. B and C keep adding each other forever.

Why? 'B': ['C'] means every time we expand B, C goes into the queue.

'C': ['B'] means every time we expand C, B goes into the queue.

Without anything to stop it, this runs forever.

Expected output when you run it

```
Step 1: Visiting S
Step 2: Visiting A
Step 3: Visiting B
Step 4: Visiting C
Step 5: Visiting C ← C again!
Step 6: Visiting B ← B again!
Step 7: Visiting B ← B again!
```

Step 8: Visiting C ← C again!

Stopped — infinite loop!

Part 3: BFS on Graph with visited set → fixed

Run BFS on the graph again, now **with** a visited set. Search for C.

Step-by-step queue trace

Step	Node	In visited?	What happens	Queue after	Visited set
1	S	No	Process it → add A, B	[A, B]	{S}
2	A	No	Process it → add C	[B, C]	{S, A}
3	B	No	Process it → add C	[C, C]	{S, A, B}
4	C	No	Process it → B is visited, skip	[C]	{S, A, B, C}
5	C	YES	SKIP — do nothing	[]	{S, A, B, C}

What you observe

Step 5 is the key moment: C appears in the queue a second time.

The visited set says 'already seen' — so we skip it completely.

Skipping that one duplicate is enough to break the B↔C cycle.

BFS finishes cleanly, just like it did on the tree.

Expected output when you run it

Visiting: S

Visiting: A

Visiting: B

Visiting: C

(Skipping C — already visited) ← this one line breaks the cycle

Goal found! Path: S → A → C

Summary

	Tree	Graph (with B↔C cycle)
Back-edges in data?	No	Yes (B↔C)
BFS without visited set	Works fine	Infinite loop
BFS with visited set	Works (fine too)	Works correctly
Why?	No cycle in structure	Visited set skips B/C

Key insight: Adding a visited set makes graph BFS **behave like** tree BFS: each node processed once. But the back-edges **still exist** in the graph data. The visited set is just your code choosing not to follow them a second time.

Task 3: Depth-First Search (DFS) for Deep Exploration

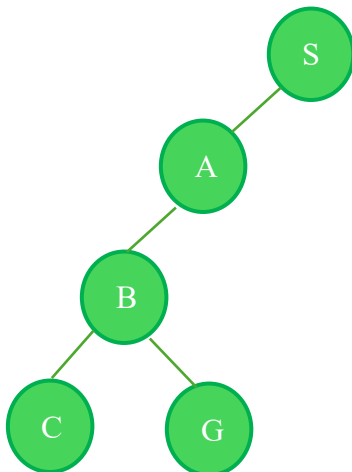
Objective

Implement **Depth-First Search** to explore maximal depth paths before backtracking.

Problem Description

Apply DFS to traverse a graph that may contain cycles and multiple solution paths, emphasizing **deep state exploration**.

Graph for DFS



- **Initial State:** S
- **Goal State:** G

Implementation Requirements

- Use either:
 - Recursive function call stack, or
 - Explicit LIFO stack
- Maintain visited states to avoid infinite loops
- Capture and display the order of node expansion

Expected Computational Behavior

- Not optimal
- Not guaranteed to find the shallowest solution

Task 4: Uniform Cost Search (UCS) for Optimal Path Discovery

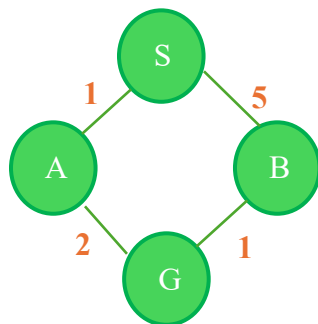
Objective

Implement **Uniform Cost Search** to identify the least-cost path in a weighted search space.

Problem Description

Apply UCS to a weighted graph where the cost of reaching a state is the cumulative sum of edge weights.

Graph for UCS



Initial State: S

Goal State: G

Implementation Requirements

- Use a **priority queue (min-heap)** ordered by path cost

- Store cumulative cost for each frontier node
- Apply goal test only when a node is dequeued from the priority queue
- Prevent re-expansion of higher-cost paths

Expected Computational Behavior

- Complete
- Optimal (given non-negative costs)

UNGRADED HOME TASK:

Task 5: Breadth-First Search (BFS) for Shallowest-Solution Discovery

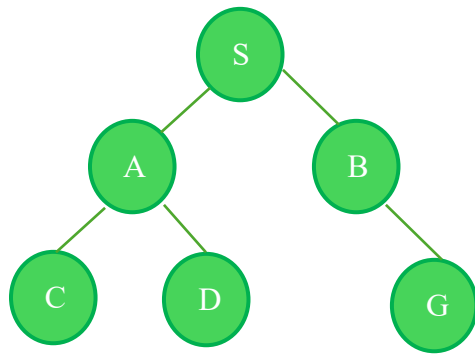
Objective

Implement **Breadth-First Search** to systematically explore the search space level by level.

Problem Description

Apply BFS to identify the **minimum-depth solution path** from the initial state to the goal state in an **unweighted graph**.

Graph for BFS



- **Initial State:** S
- **Goal State:** G

Implementation Requirements

- Use a **FIFO queue** for frontier management
- Maintain an explored set to prevent redundant state expansion
- Record parent pointers to reconstruct the solution path

- Terminate search immediately upon goal detection

Expected Computational Behavior

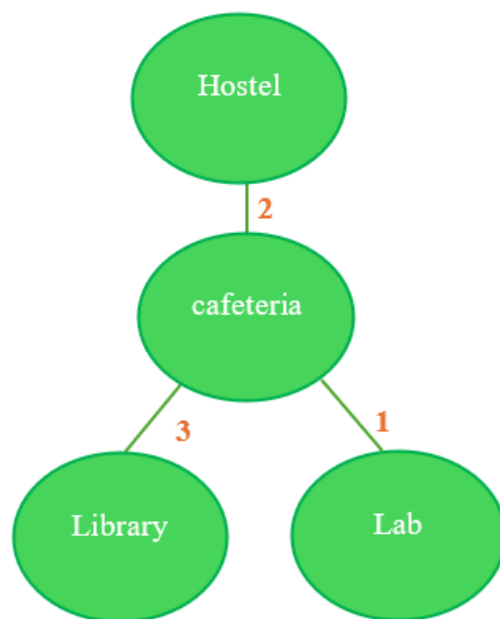
- Complete
- Optimal with respect to path length (number of edges)

Task 6: Autonomous Campus Navigation Agent

Objective

Apply uninformed search techniques to a **realistic navigation problem**.

Graph Representation



Implementation Requirements

- BFS: Find minimum number of locations visited
- DFS: Explore possible navigation routes
- UCS: Determine minimum travel distance

Expected Outcome

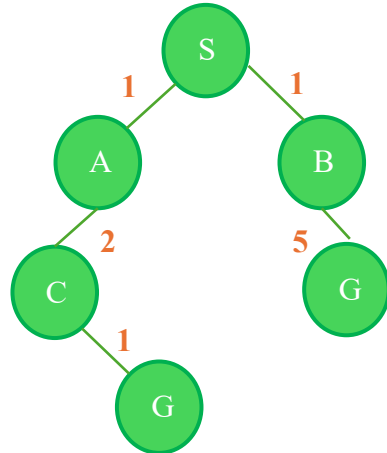
- Practical interpretation of abstract search concepts
- Clear distinction between depth-based, breadth-based, and cost-based search strategies

Task 7: Comparative Performance Analysis of BFS, DFS, and UCS

Objective

Evaluate and compare uninformed search strategies based on computational properties.

Common Graph for Comparison



Analysis Parameters

You must analyze:

- Number of nodes expanded
- Path length
- Path cost
- Completeness
- Optimality

Implementation Requirements

- Execute all three algorithms on the same graph
- Log:
 - Node expansion sequence
 - Final solution path
 - Total path cost (where applicable)

*Note: In all tasks, Visualize each graph using **networkx** Library.*

Submission Guidelines:

1. Prepare a **Word report** containing screenshots of the output for **all tasks**.
2. Ensure that each screenshot is **clearly placed under its respective task**.
3. **Download the .ipynb file** from Jupyter Notebook.
4. **Upload the Word report, and .ipynb** to GCR. **Do NOT** upload a ZIP file.
5. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**